Data Cleaning and Preprocessing

Week 9

Nurul Afiqah Burhanuddin
nurul.afiqah@ukm.edu.my

Introduction

Some data cleaning and preprocessing steps:

- Handling duplicates
- Handling missing data
- Replacing values
- Data transformation using function or mapping
- Handling outliers
- Discretization
- Permutation and random sampling
- Dummy variables
- String manipulation
- Regular expressions

Handling Duplicates

- Checking for duplicates:

```
df.duplicated():
```

  - returns a boolean Series indicating if each row is a duplicate of a previous row

- Removing duplicates:

```
drop_duplicates()
```

  - subset: Specify columns to check for duplicates.
  - keep: Determines which duplicate to keep: first, last, or none.

Handling Missing Data

- In Pandas, missing data is represented by `np.nan` and `None`.
- Detecting missing data:
  - `.isna()` or `.isnull()`: Returns a DataFrame of the same shape with True for missing values and False for non-missing values.
  - `.notna()` or `.notnull()`: Returns the inverse (non-missing values as True)

Handling Missing Data

- Dropping missing data:
  - `df.dropna(axis=0)`: Drops rows with missing values
  - `df.dropna(axis=1)`: Drops columns with missing values
  - `df.dropna(subset=['column'], axis=0)`: Drop rows with missing values in a specific column

Handling Missing Data

- Filling missing data:
  - `df.fillna()`: Fill missing data with a specified value
  - `df.ffill()`: Forward fill (use previous value)
  - `df.bfill()`: Backward fill (use next value)
  - `df.interpolate(method='linear')`: Fill by linear interpolation (for numeric columns)

Replacing Values

The `replace()` function in Pandas is versatile and can handle:

- Direct replacements

```
df.replace(replace, value)
```

- Multiple replacements - You can replace multiple values by passing a list or dictionary.

```
df.replace([replace_1, replace_2], value)
```

```
df.replace({replace_1:value 1, replace_2:value_2})
```

Data Transformation Using Function/Mapping

- `apply()`
  - Used for row-wise or column-wise transformations in a DataFrame.
  - Can apply a custom function to entire rows or columns.
  - Allows for complex transformations based on multiple values in a row/column.
- `map()`
  - Used for element-wise transformations.
  - Applies a function to each element of the Series.
  - Can also map values using a dictionary - useful for categorical transformations.

Handling Outliers

- Filtering outliers:

```
df[abs(df['col'])> threshold]
```

- Replace with a specific value:
  - Use loc for conditional replacement
  - Use `apply()` function to apply a custom function for outlier replacement logic
  - Use `np.where()` for conditional replacement

Discretization

- Divides data into bins with equal ranges (intervals) or specified intervals.

```
pd.cut(x, bins, labels=None, right=True)
```

- Divides data into bins with equal frequencies (quantiles).

```
pd.qcut(x, q, labels=None)
```

Permutation and Random Sampling

- Permutation: Rearranging the elements of a dataset in a random order.
  - `numpy.random.permutation()`
  - `DataFrame.sample(frac=1)`
- Random Sampling: Selecting a subset of data from a larger dataset, either with or without replacement.
  - `numpy.random.choice()`
  - `DataFrame.sample()`

Dummy Variables

- Dummy Variables are binary (0 or 1) variables used to represent categorical data in numerical form.
- They are widely used in machine learning and statistical modeling to encode categorical data.
- Each category in a feature gets its own column, with 1 indicating the presence of that category and 0 indicating its absence.

```
pd.get_dummies(data, columns=None, drop_first=False)
```

String Manipulation

- In Python, string objects come with a wide range of built-in methods useful for performing common string manipulation tasks.

```
str.method()
```

- These methods are useful for quick operations on individual string.

String Manipulation

Method	Description
`.lower()`	Converts all characters to lowercase
`.upper()`	Converts all characters to uppercase
`.capitalize()`	Converts the first character to uppercase
`.title()`	Converts the first character of each word to uppercase
`.isdigit()`	Checks if all characters are digits
`.isalpha()`	Checks if all characters are alphanumeric
`.isspace()`	Checks if all characters are whitespace
`.isupper()`	Checks if all characters are uppercase
`.islower()`	Checks if all characters are lowercase
`.istitle()`	Checks if the string is in title case

String Manipulation

Method	Description
`.startswith(sub)`	Checks if the string starts with sub
`.endswith(sub)`	Checks if the string ends with sub
`.contains(sub)`	Checks if the string contains sub
`.find(sub)`	Finds the first occurrence of sub
`.rfind(sub)`	Finds the last occurrence of sub
`.index(sub)`	Like .find(), but raises an error if not found
`.strip()`	Removes whitespace
`.lstrip()`	Removes leading whitespace
`.rstrip()`	Removes trailing whitespace
`.split(sep)`	Splits string into a list by sep
`.join(iterable)`	Joins elements of iterable with the string
`.replace(old, new)`	Replaces occurrences of old with new

String Manipulation

- In Pandas, instead of working on individual strings, you can apply string operations to entire columns or Series using the .str attribute.

```
df['column'].str.method()
```

- This allows efficient, vectorized operations for string manipulation on all elements of the Series or DataFrame column.

Regular Expressions

- Regular Expressions (regex) are powerful tools for matching, searching, and manipulating strings based on patterns.
- Regex defined as a sequence of characters that represents a search pattern.
- Python provides regex functionality through the `re` module.
- Regex patterns often contain backslashes (\). To avoid escaping them in Python strings, use raw strings

```
r'...'
```

Regular Expression

Some common functions in the re module:

Method	Description
`re.match()`	Checks if the regex pattern matches at the start of the string.
`re.search()`	Searches for the first occurrence of the regex pattern anywhere in the string.
`re.findall()`	Returns a list of all non-overlapping matches of the pattern in the string.
`re.finditer()`	Like findall but returns an iterator
`re.sub()`	Replaces all occurrences of the pattern with the specified replacement string.
`re.split()`	Splits the string by the occurrences of the pattern

Regular Expression

Predefined classes of characters commonly appear in regular expressions:

Class	Description
`\d`	Matches any digit (0-9)
`\D`	Matches any non-digit
`\w`	Matches any word character (letters, digits, or underscore)
`\W`	Matches any non-word character
`\s`	Matches any whitespace character
`\S`	Matches any non-whitespace

Regular Expression

Some special character in regex:

Character	Description
`.`	Matches any character (except newline)
`^`	Matches the start of the string
`$`	Matches the end of the string
`*`	Matches 0 or more repetitions
`+`	Matches 1 or more repetitions
`?`	Matches 0 or 1 repetition
`{n}`	Matches exactly n repetitions
`{n,m}`	Matches between n and m repetitions
`[]`	Matches any characters listed inside the brackets
`[^]`	Matches any characters not listed inside the brackets
`\`	Matches either pattern
`()`	Groups patterns
`\`	Escape special characters or use predefined character classes

Regular Expression

Some special character in regex:

Character	Description
`.`	Matches any character (except newline)
`^`	Matches the start of the string
`$`	Matches the end of the string
`*`	Matches 0 or more repetitions
`+`	Matches 1 or more repetitions
`?`	Matches 0 or 1 repetition
`{n}`	Matches exactly n repetitions
`{n,m}`	Matches between n and m repetitions
`[]`	Matches any characters listed inside the brackets
`[^]`	Matches any characters not listed inside the brackets
`\`	Matches either pattern
`()`	Groups patterns
`\`	Escape special characters or use predefined character classes